

GNN Results Summary — 2-Gene & 3-Gene Sequential Testing POMDP

What We Fixed (PI Feedback)

Problem: The GNN was only receiving the belief state (posterior probabilities) as input. $V^*(s)$ depends on BOTH the belief state AND the cost function parameters (a, b, delta per gene, fixed/variable test cost). Without cost params as input, a single model cannot distinguish between Base and Aggressive cost regimes — it gets contradictory labels for the same input.

Fix: Added a global cost vector concatenated to the graph embedding before the head MLP.

```
graph_repr = mean_pool(node_embeddings)      # (B, 32)
cost_vec    = [a, b, delta per gene, fixed, variable] # (B, G)
combined    = concat([graph_repr, cost_vec])   # (B, 32+G)
V*(s)       = head_MLP(combined)              # (B, 1)
```

Architecture — PedigreeGNN (shared/model.py)

Node Features (per person/node)

Genes	Features	Dim
2 genes	$P(g=0,1,2)_{\text{GeneA}} + P(g=0,1,2)_{\text{GeneB}} + \text{is_tested}$	7
3 genes	$P(g=0,1,2)_{\text{GeneA}} + P(g=0,1,2)_{\text{GeneB}} + P(g=0,1,2)_{\text{GeneC}} + \text{is_tested}$	10

Cost Vector (global, per config)

Genes	Contents	Dim
2 genes	a_A, b_A, delta_A, a_B, b_B, delta_B, fixed_cost, variable_cost	8
3 genes	a_A, b_A, delta_A, a_B, b_B, delta_B, a_C, b_C, delta_C, fixed_cost, variable_cost	11

Cost Preset Values

Param	Base	Aggressive
a_GeneA	-0.08	-0.12
b_GeneA	-0.04	-0.06
delta_GeneA	0.60	0.70
a_GeneB	-0.06	-0.09
b_GeneB	-0.03	-0.045
delta_GeneB	0.70	0.80
a_GeneC	-0.05	-0.075
b_GeneC	-0.025	-0.0375
delta_GeneC	0.75	0.80
fixed_cost	0.01	0.01
variable_cost	0.02	0.02

Message Passing

- 2 rounds of parent-to-child message passing along pedigree edges
- Per round: `msg MLP([src || dst] -> 32) + update MLP([node || agg_msg] -> 32)`
- After 2 rounds: per-node embeddings are shape (B, N, 32)
- Global mean pool -> (B, 32) graph embedding
- Concat cost vec -> head MLP: 43 -> 16 -> 1 (3-gene) / 40 -> 16 -> 1 (2-gene)

Metric: ratio2

$$\text{ratio2} = (\text{V_root} - \text{L}) / (\text{V_root} - \text{V_stop})$$

- **V_root** : exact DP optimal value from root state
 - **V_stop** : value of stopping immediately (no tests)
 - **L** : value achieved by greedy GNN policy
 - **0 = GNN matches exact DP (optimal)**
 - **1 = GNN no better than stopping**
-

Train / Test Split

2-Gene (step5)

- **Train:** Extended family (LowHigh + MediumEven) x (Base + Aggressive) = 4 configs + ThreeGeneration LowHigh x (Base + Aggressive) = 2 configs **Total: 6 train configs**
- **Test:** ThreeGeneration MediumEven x (Base + Aggressive) = **2 held-out configs**

3-Gene (step9)

- **36 total configs** = 3 families x 6 allele regimes x 2 presets
- **Train: 30 configs** (5 regimes x 2 presets x 3 families)
- **Test: 6 held-out configs** — MediumEven_Aggressive + HighHigh_Aggressive for each family

Families: Trio (N=3), Nuclear (N=4), ThreeGeneration (N=5)

Allele regimes: LowHigh, MediumEven, LowLow, HighHigh, MixedA, MixedB

Results — 2-Gene GNN (step5, 500 epochs, CPU)

Family	Regime	Preset	Split	ratio2
Extended	LowHigh	Base	TRAIN	0.030359
Extended	LowHigh	Aggressive	TRAIN	0.008468
Extended	MediumEven	Base	TRAIN	0.026152
Extended	MediumEven	Aggressive	TRAIN	0.009894
ThreeGeneration	LowHigh	Base	TRAIN	0.022874
ThreeGeneration	LowHigh	Aggressive	TRAIN	0.003153
ThreeGeneration	MediumEven	Base	TEST	0.013205
ThreeGeneration	MediumEven	Aggressive	TEST	0.001555

Results — 3-Gene GNN (step9, 50 epochs / 500, GPU)

Test Configs (held out — never seen during training)

Family	Regime	Preset	Split	ratio2
Nuclear	MediumEven	Aggressive	TEST	0.000018
ThreeGeneration	HighHigh	Aggressive	TEST	0.000262
Nuclear	HighHigh	Aggressive	TEST	0.003014
ThreeGeneration	MediumEven	Aggressive	TEST	0.005452
Trio	HighHigh	Aggressive	TEST	0.014031
Trio	MediumEven	Aggressive	TEST	0.016770

All 36 Configs (sorted by ratio2)

Split	Family	Regime	Preset	ratio2
TRAIN	Trio	LowLow	Base	1.000000
TRAIN	Trio	LowHigh	Base	0.120612
TRAIN	Trio	MixedB	Base	0.065461
TRAIN	Trio	MixedA	Base	0.060488
TRAIN	Trio	MediumEven	Base	0.054671
TRAIN	ThreeGeneration	LowLow	Base	0.038045
TEST	Trio	MediumEven	Aggressive	0.016770
TRAIN	Trio	MixedA	Aggressive	0.014540
TEST	Trio	HighHigh	Aggressive	0.014031
TRAIN	Trio	MixedB	Aggressive	0.013083
TRAIN	Trio	LowHigh	Aggressive	0.011718
TRAIN	Nuclear	MixedA	Base	0.011035
TRAIN	ThreeGeneration	MixedB	Aggressive	0.010482
TRAIN	Nuclear	LowHigh	Base	0.008048
TRAIN	ThreeGeneration	LowLow	Aggressive	0.007052
TRAIN	Nuclear	MixedB	Base	0.006718
TRAIN	Nuclear	MixedB	Aggressive	0.006162
TRAIN	Trio	HighHigh	Base	0.006088
TEST	ThreeGeneration	MediumEven	Aggressive	0.005452
TRAIN	ThreeGeneration	MediumEven	Base	0.005333
TRAIN	ThreeGeneration	MixedA	Aggressive	0.004141
TRAIN	ThreeGeneration	MixedB	Base	0.003691
TRAIN	ThreeGeneration	LowHigh	Aggressive	0.003174
TRAIN	ThreeGeneration	MixedA	Base	0.003027
TEST	Nuclear	HighHigh	Aggressive	0.003014
TRAIN	Nuclear	HighHigh	Base	0.002756

Split	Family	Regime	Preset	ratio2
TRAIN	ThreeGeneration	LowHigh	Base	0.001614
TRAIN	Nuclear	MediumEven	Base	0.001067
TRAIN	Nuclear	LowLow	Base	0.000714
TRAIN	ThreeGeneration	HighHigh	Base	0.000419
TRAIN	Nuclear	LowLow	Aggressive	0.000347
TEST	ThreeGeneration	HighHigh	Aggressive	0.000262
TRAIN	Nuclear	LowHigh	Aggressive	0.000057
TRAIN	Nuclear	MixedA	Aggressive	0.000047
TRAIN	Trio	LowLow	Aggressive	0.000038
TEST	Nuclear	MediumEven	Aggressive	0.000018

Notes

Trio_LowLow_Base ratio2=1.0 (anomaly): $V_{\text{root}} - V_{\text{stop}} = 0.011$ — the margin from testing is extremely small (all 3 genes very rare, $q=0.02$). GNN policy stops at root rather than testing. This is a class-imbalance artifact (Trio = 0.44% of training data vs ThreeGeneration = 95.6%) compounded by the tiny reward gap. Not a code bug. Would improve with class-balanced training or more epochs.

Only 50 / 500 epochs for 3-gene — wall time hit at 4h. Nuclear and ThreeGeneration results are strong at epoch 50. Trio would improve with longer training and balanced sampling.

GeneC parameters are placeholders — not yet confirmed by PI.

Files Changed

File	Change
shared/model.py	Added <code>global_feat_dim</code> param; head takes <code>[graph_repr \ \ cost_vec]</code>
step5_gnn/run.py	Added <code>config_to_cost_vec()</code> ; cost vec passed through training + eval
step9_gnn_3gene/run.py	Same; cost vec tiled per config and stored as <code>gf</code> in training groups
step9_gnn_3gene/eval.py	<code>precompute_gnn_values()</code> tiles cost vec; passes <code>gf_b</code> to <code>forward_batch</code>
submit_step9_eval_auto.sh	Added <code>--fresh</code> flag to avoid reusing stale cached results

Files Where GNN and Embeddings Are Defined

File	What it contains
<code>shared/model.py</code>	PedigreeGNN class — full architecture, message passing, embedding computation
<code>step9_gnn_3gene/run.py</code>	Training loop; <code>extract_node_features()</code> builds per-node feature tensors
<code>step9_gnn_3gene/eval.py</code>	<code>precompute_gnn_values()</code> — runs forward pass, extracts per-state GNN predictions
<code>step5_gnn/run.py</code>	2-gene version; <code>state_to_graph()</code> builds node features per state

Node embeddings are the intermediate `h` tensor inside `PedigreeGNN._mp_step()`: shape `(B, N, 32)` after each message-passing round. The graph embedding (before the scalar head) is `h.mean(dim=1)` — shape `(B, 32)`. These are computed on-the-fly; to extract them, call `model.forward_batch()` and hook the mean-pool step.